

B Decision-making

Environment: You are in the middle of a room. Looking quickly around you, you see a bed 1, a desk 2, a desk 1, a drawer 6, a drawer 5, a drawer 4, a drawer 3, a drawer 2, a drawer 1, a garbagecan 1, a laundryhamper 1, a safe 1, a shelf 6, a shelf 5, a shelf 4, a shelf 3, a shelf 2, and a shelf 1.

Task: examine the mug with the desklamp.

Trial #1

> think: To solve the task, I need to find and take a mug, then find and use a desklamp.
> think: First I need to find a mug. A mug is more likely to appear in drawer (1-6), desk (1-2), shelf (1-6), garbagecan (1), laundryhamper (1). I can check one by one, starting with drawer 1.
> go to drawer 1
The drawer 1 is closed.

...

> open drawer 6
You open the drawer 6. The drawer 6 is open. In it, you see a keychain 2.

> go to desk 1

On the desk 1, you see a creditcard 3, a desklamp 1, a laptop 2, a mug 1, a pen 1, and a pencil 1.

> think: Now I find a mug (1). Next, I need to take it.

> take mug 1 from desk 1

You pick up the mug 1 from the desk 1.

> think: Now I take a mug (1). Next, I need to find a desklamp. A desklamp is more likely to appear in desk (1-2), sidetable (1-2), shelf (1-6), bed (1), drawer (1-6). I can check one by one, starting with desk 1.

> go to desk 1

Nothing happens.

> go to desk 2

On the desk 2, you see a alarmclock 1, a bowl 1, a mug 3, a pencil 3, and a pencil 2.

> think: Now I find a desklamp (1). Next, I need to use it.

> use desklamp 1

Nothing happens.

> use desklamp 1

Nothing happens.

Status: Fail

Reflection: In this environment, my plan was to find a mug then find and use a desklamp. However, the task says to examine the mug with the desklamp. I should have looked for the desklamp first, then looked for the mug. I noticed that the desklamp was found on desk 1. In the next trial, I will go to desk 1, find the lamp, then look for the mug and examine it with the desklamp.

Trial #2

> go to desk 1

On the desk 1, you see a creditcard 3, a desklamp 1, a laptop 2, a mug 1, a pen 1, and a pencil 1.

> think: To solve the task, I need to find and take a mug, then find and use a desklamp.

> take mug 1 from desk 1

You pick up the mug 1 from the desk 1.

> think: To solve the task, I need to find and take a mug, then find and use a desklamp.

> use desklamp 1

You turn on the desklamp 1.

Status: Success

Figure 5: [Top] An AlfWorld trajectory in which the agent failed due to inefficient planning. In the reflection, the agent recognizes that it should have looked for the desklamp then the mug, not the mug then the desklamp. [Bottom] The agent is able to correct its reasoning trace and execute a sequence of actions in a concise manner.

B.1 WebShop Limitation

In 5, we briefly state that Reflexion struggles to overcome local minima choices that require extremely creative behavior to escape. We observe this shortcoming in an experiment on WebShop [29]. WebShop is a web-based problem-solving benchmark that tests agents to navigate an e-commerce website to locate and purchase products given requests from clients. We test a two-shot ReAct + Reflexion agent in 100 environments. However, after only four trials, we terminate the runs as the agent does not show signs of improvement 6. Further, the agent does not generate helpful, intuitive self-reflections after failed attempts. We conclude that Reflexion is unable to solve tasks that require a significant amount of diversity and exploration. In AlfWorld, the agent is able to adequately explore new environments because the permissible actions can be seen in the observations. In HotPotQA, the agent faces a similar WebShop search query task but is more successful as the search space for Wikipedia articles is more diverse and requires less precise search queries. A common problem for e-commerce search engines is properly handling ambiguity in natural language search interpretations. Thus, WebShop presents a task that requires very diverse and unique behavior from a Reflexion agent.

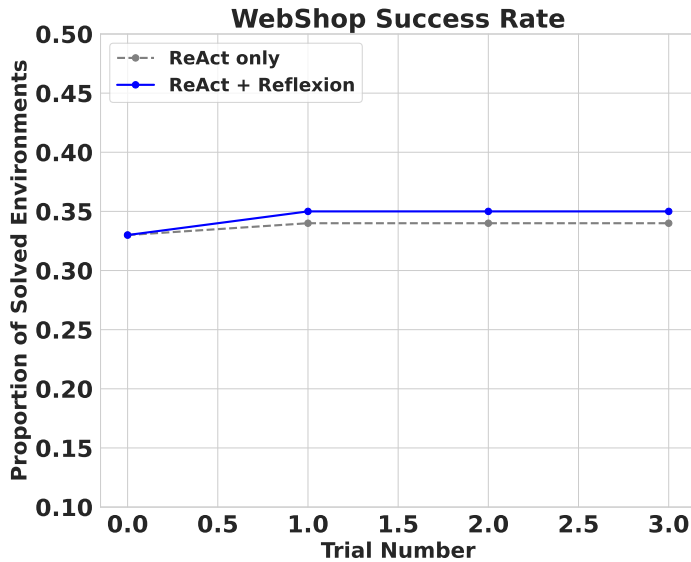


Figure 6: Reflexion vs React performance on WebShop across 100 customer shopping requests. ReAct + Reflexion fails to significantly outperform ReAct.

C Programming

Programming LLM calls require strict instructions to produce function bodies *only*, due to the extensive dialogue training of the LLMs. A few programming examples are reported below with instructions **highlighted in blue** and templates. See the full implementation at <https://github.com/noahshinn024/reflexion>.

C.1 Programming function implementation example (HumanEval Python)

Sample function signature:

```
1 def minSubArraySum(nums):
2     """
3     Given an array of integers nums, find the minimum sum of
4     any
5     non-empty sub-array of nums.
6     Example
7     minSubArraySum([2, 3, 4, 1, 2, 4]) == 1
```

```

7 minSubArraySum([-1, -2, -3]) == -6
8 """

```

Sample function body implementation:

```

1 min_sum = float('inf')
2 for i in range(len(nums)):
3     current_sum = 0
4     for j in range(i, len(nums)):
5         current_sum += nums[j]
6         if current_sum < min_sum:
7             min_sum = current_sum
8 return min_sum

```

C.2 Reflexion Actor instruction

You are a Python writing assistant. You will be given your previous implementation of a function, a series of unit tests results, and your self-reflection on your previous implementation. Apply the necessary changes below by responding only with the improved body of the function. Do not include the signature in your response. The first line of your response should have 4 spaces of indentation so that it fits syntactically with the user provided signature. You will be given a few examples by the user.

Reflexion Actor generations follow the form:

(Instruction)

(Function implementation)

(Unit test feedback)

(Self-reflection)

(Instruction for next function implementation)

Full examples are shown in <https://github.com/noahshinn024/reflexion>

C.3 Reflexion Self-reflection instruction and example

You are a Python writing assistant. You will be given your previous implementation of a function, a series of unit tests results, and your self-reflection on your previous implementation. Apply the necessary changes below by responding only with the improved body of the function. Do not include the signature in your response. The first line of your response should have 4 spaces of indentation so that it fits syntactically with the user provided signature. You will be given a few examples by the user. Reflexion Self-Reflection generations follow the form:

(Instruction)

(Function implementation)

(Unit test feedback)

C.4 Reflexion programming *no* Self-Reflection ablation example

Reflexion *no* Self-Reflection ablation Actor generations follow the form:

(Instruction)

(Function implementation)

(Unit test feedback)

(Self-reflection)

(Instruction for next function implementation)

C.5 Reflexion programming *no* test generation ablation example

Reflexion *no* test generation ablation Actor generations follow the form:

(Instruction)